

SWE 4503

Web Hackery

Introduction

The ability to analyze web applications is an absolutely critical skill for any attacker or penetration tester. Quite frankly, topics such as SQL injection have been beaten to death, and the tooling available is mature enough that we don't need to reinvent the wheel. Instead, we'll explore the basics of interacting with the web by using Python and then build on this knowledge to create reconnaissance and brute-force tooling. In this chapter, we'll look at three scenarios for attacking a web app.

Scenarios

1. In the first scenario, you know the web framework that the target uses, and that framework happens to be open source. A web app framework contains many files and directories within directories within directories. We'll create a map that shows the hierarchy of the web app locally and use that information to locate the real files and directories on the live target.
2. In the second scenario, you know only the URL for your target, so we'll resort to brute-forcing the same kind of mapping by using a word list to generate a list of file paths and directory names that may be present on the target. We'll then attempt to connect to the resulting list of possible paths against a live target.
3. In the third scenario, you know the base URL of your target and its login page. We'll examine the login page and use a word list to brute-force a login.

Simple Get Request and Response

1. **Import Modules** : We import urllib.request for making the request and json if you want to parse the response (optional).
2. **URL** : We're using `http://httpbin.org/get` which is a dummy endpoint that returns details about your GET request. `httpbin.org` is a free HTTP request and response testing service. It allows you to test HTTP methods like GET, POST, PUT, DELETE, and more by providing mock endpoints.
3. **Sending Request** : `urllib.request.urlopen(url)` sends the GET request to the URL.
4. **Reading and Decoding Response** : `response.read()` gets the response data in bytes, and `.decode('utf-8')` converts it to a string.
5. **Output** : The printed output will be a JSON formatted string showing details of the request.

```
import urllib.request
import json

url = 'http://httpbin.org/get'
response = urllib.request.urlopen(url)
data = response.read().decode('utf-8')
print(data)
response.close()
```

Execution: Just run the above code with `python simple_get_request.py` And make sure you have `python3.11.9`

Get Request with Customized User Agent

A customized user agent can offer several benefits:

- **Bypassing Blocks:** Some servers restrict or block requests from known automated clients or outdated browsers. Spoofing a common user agent can help bypass these filters.
- **Content Optimization:** Websites may serve different content based on the client. A custom user agent lets you request the version optimized for a specific browser or device.
- **Debugging and Testing:** During development, setting a custom user agent allows you to simulate requests from different browsers or devices to test how your web service responds.
- **Analytics and Logging:** Some APIs or web services use the user agent to track usage. Customizing it can help identify requests coming from your application versus others.

Execute the right side code the same way you have implemented the previous code

```
1  import urllib.request
2  import json
3
4  url = 'http://httpbin.org/get'
5  headers = {'User-Agent': 'Mozilla/5.0 (compatible; MyApp/1.0)'}
6
7  req = urllib.request.Request(url, headers=headers)
8  try:
9      with urllib.request.urlopen(req) as response:
10         data = response.read().decode('utf-8')
11         print(json.loads(data))
12 except Exception as e:
13     print(f"Found {e}")
```

Post Request

1. First we import required modules.
2. Give our endpoint.
3. Create a dictionary since endpoint data as a dictionary.
4. Encode the data in bytes.
5. Post the data and print the response of the endpoint.

```
1 import urllib.request
2 import urllib.parse
3 url = "https://httpbin.org/post"
4 data = {
5     "name": "John",
6     "age": 30
7 }
8 encoded_data = urllib.parse.urlencode(data).encode("utf-8")
9 req = urllib.request.Request(url, data=encoded_data, method="POST")
10 req.add_header("Content-Type", "application/x-www-form-urlencoded")
11 with urllib.request.urlopen(req) as response:
12     result = response.read().decode("utf-8")
13 print(result)
```

LXML and REQUESTS Module

1. Request Module give us more tools than built-in urllib module. For an example, requests module automatically handles cookies for me.
2. LXML is parsing module that will parse the response to extract important information. For an instance, printing all the anchor with links and their http address.

In the right side code,

1. Content is byte string.
2. Parser from lxml can't work with raw byte string. That's why we need to encapsulate it with BytesIO class to make it as a file like object (Simply a file).

```
1  from io import BytesIO
2  from lxml import etree
3  import requests
4  url = 'https://nostarch.com'
5  r = requests.get(url)
6  content = r.content
7  parser = etree.HTMLParser()
8  content = etree.parse(BytesIO(content), parser=parser)
9  for link in content.findall('//a'):
10     print(f"{link.get('href')} -> {link.text}")
```

Mapping Open Source Web App Installations

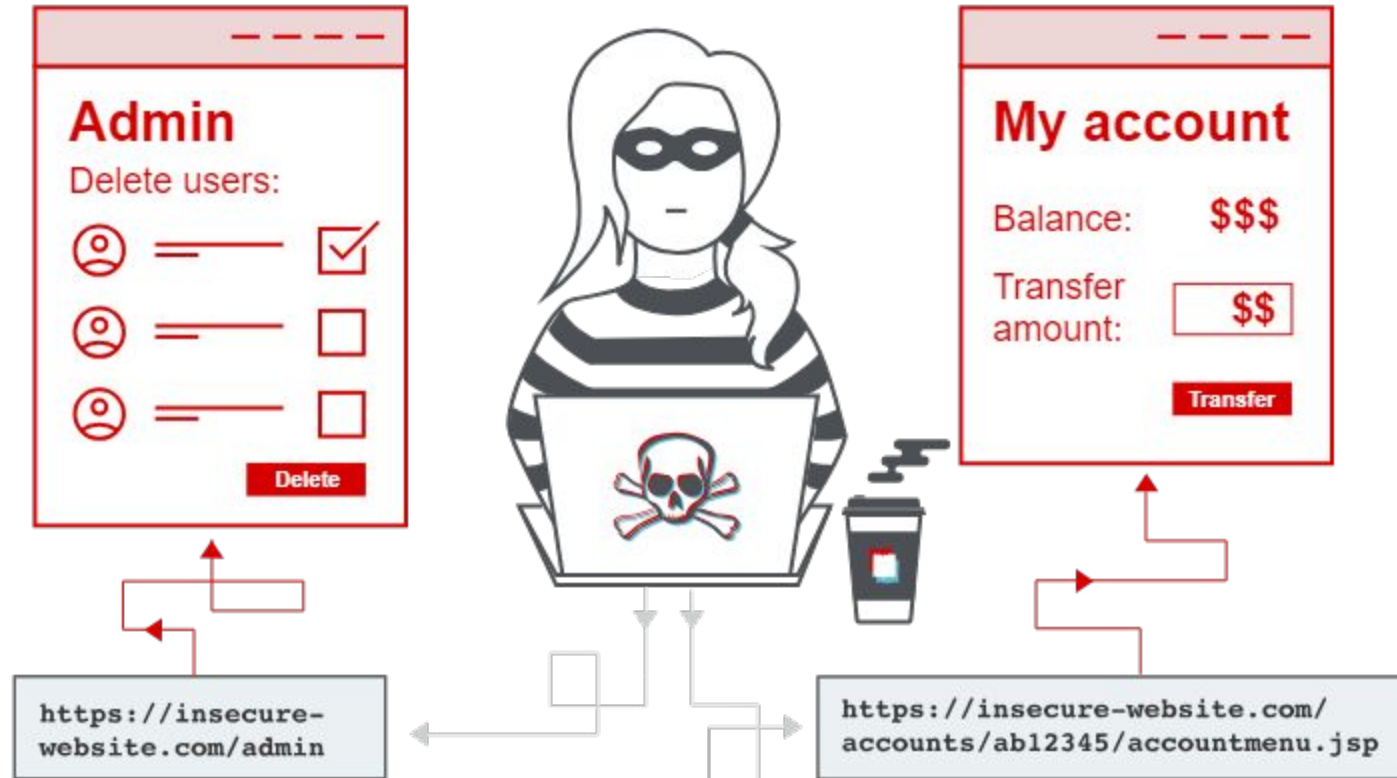
Content management systems (CMSs) and blogging platforms such as Joomla, WordPress, and Drupal make starting a new blog or website simple, and they're relatively common in a shared hosting environment or even an enterprise network. All systems have their own challenges in terms of installation, configuration, and patch management, and these CMS suites are no exception. When an overworked sysadmin or a hapless web developer doesn't follow all security and installation procedures, it can be easy pickings for an attacker to gain access to the web server.

As for example, lets say an inexperienced web developer is working on webhackery.com and we can log in to our admin page from webhackery.com/home page. The address of admin page is webhackery.com/admin. Since he did not implemented enough secure procedures so any one directly searching for webhackery.com/admin can directly access the admin page. We can dive further into next few slides of access control.

Access Control

- determines whether the user is allowed to carry out the action that they are attempting to perform.
- Authentication and Session management.
- Vertical and Horizontal privilege escalation. If attacker tries to access admin it is vertical privilege escalation. If attacker tries to access users it is horizontal privilege escalation.

Illustration of Broken Access Control



Vertical Privilege Escalation

- We use different files to help crawlers.
- `https://insecure-website.com/admin`
- `https://insecure-website.com/robots.txt`
- wordlist to brute-force
- [LINKforLAB](#)
- Solution can be using uncommon url extension like :
`https://insecure-website.com/administrator-panel-yb556`

```
import requests
```

```
target = "https://www.nytco.com/"
```

```
wp_paths = [  
    "wp-admin/", "wp-content/", "wp-includes/", "wp-config.php", "readme.html",  
    "xmlrpc.php", "wp-login.php", "wp-cron.php", "wp-signup.php",  
    "wp-links-opml.php", "wp-trackback.php"  
]
```

A simple mapper

```
def check_paths(target, paths):  
    print(f"Scanning {target} for standard WordPress directories and files...\n")
```

```
    for path in paths:
```

```
        url = f"{target}/{path}"
```

```
        try:
```

```
            response = requests.get(url, timeout=5)
```

```
            if response.status_code == 200:
```

```
                print(f"[+] Found: {url}")
```

```
            elif response.status_code == 403:
```

```
                print(f"[!] Forbidden: {url} (Access Denied)")
```

```
            else:
```

```
                print(f"[-] Not Found: {url}")
```

```
        except requests.exceptions.RequestException:
```

```
            print(f"[!] Error accessing {url}")
```

```
check_paths(target, wp_paths)
```

Scanning https://www.nytco.com/ for standard WordPress d

[+] Found: https://www.nytco.com//wp-admin/

[-] Not Found: https://www.nytco.com//wp-content/

[-] Not Found: https://www.nytco.com//wp-includes/

[+] Found: https://www.nytco.com//wp-config.php

[-] Not Found: https://www.nytco.com//readme.html

[-] Not Found: https://www.nytco.com//xmlrpc.php

[-] Not Found: https://www.nytco.com//wp-login.php

[-] Not Found: https://www.nytco.com//wp-cron.php

[-] Not Found: https://www.nytco.com//wp-signup.php

[-] Not Found: https://www.nytco.com//wp-links-opml.php

[-] Not Found: https://www.nytco.com//wp-trackback.php

Local WordPress Site

It is not ethical to try and hack others WordPress site. That's why we need our own local WordPress site using which we can practise our attacks. And by practising our attacks we can understand what to do when we have to defense.

We can easily make and launch a wordpress site locally utilizing [localwp](#). We also need to download [wordpress](#) folder to first understand what might be the structure of a wordpress website.

So right now I am serving a wordpress blog site called webHackery in my Host os windows. And also running my python code link_parser.py in the same host os windows (fortunately no kali till now). The link_parser.py is the same code where I was showing how to use lxml and requests module. The output looks like the right side image.

[illegible]

Mapping the WordPress Framework (1st Scenario)

Still we are using only our host os windows. My python version in windows is 3.9.6

Before going into deep let's cover some python basics. At first, **is there any keyword other than try and except for error handling?**

Yes, there are else and finally.

```
try:
    something_that_might_cause_an_error()
except SomeError as e:
    print(e)                # show the error on the console
    dosomethingelse()        # take some alternative action
else:
    everything_is_fine()     # this executes only if the try succeeded
finally:
    cleanup()                # this executes no matter what
```

The real mapper.py

Here we are importing necessary modules and defining other global variables.

We will be using 10 threads to make the process faster. Our target is the local wordpress site. We will not be processing images and style sheets since utilizing those we might not be able to pivot an attack.

```
import contextlib
import os
import queue
import requests
import sys
import threading
import time
```

```
FILTERED = [".jpg", ".gif", ".png", ".css"]
TARGET = "http://webhackery.local/"
THREADS = 10
```

```
answers = queue.Queue()
web_paths = queue.Queue()
```

Context Manager

Context managers provide a cool programming pattern, especially if you're forgetful or just have too much to keep track of and want to simplify your life. You'll find them helpful when you've opened something and need to close it, locked something and need to release it, or changed something and need to reset it. You're probably familiar with built-in file managers like open to open a file or socket to use a socket.

When we call chdir with "with" keyword it gets into the state where cwd is the path we just called chdir with and when we are getting outside the with block we are executing finally block inside chdir so that we can get back to the previous cwd/current working directory.

```
50 @contextlib.contextmanager
51 def chdir(path):
52     """
53     On enter, change directory to specified path.
54     On exit, change directory back to original.
55     """
56     this_dir = os.getcwd()
57     os.chdir(path)
58     try:
59         yield
60     finally:
61         os.chdir(this_dir)

63 if __name__ == '__main__':
64     with chdir("./chapter5\wordpress\wordpress"):
65         gather_paths()
```

Gathering Paths

1. If we refer to previous slide we can understand that when we are executing `gather_paths()`, our cwd is the path where our downloaded wordpress folder resides. We will parse this folder using `os.walk('.')` in line 40.
2. Next in line 42 I am checking if files are images or style sheets. If they are, we are avoiding those paths.
3. In line 44 we are getting the actual path by joining the root path. Next we are removing the `'.'` that means we are in cwd in line 45-46 and printing and putting paths inside the `web_paths` queue in line 47-48.

```
39 def gather_paths():
40     for root, _, files in os.walk('.'):
41         for fname in files:
42             if os.path.splitext(fname)[1] in FILTERED:
43                 continue
44             path = os.path.join(root, fname)
45             if path.startswith('.'):
46                 path = path[1:]
47             print(path)
48             web_paths.put(path)
```

Threading

1. We have 10 threads for faster processing.
2. As per line 33 each thread executes test_remote.
3. After starting all the threads in line 36 and 37 we are making sure all threads complete their execution before moving forward in the program.

```
29  def run():
30      mythreads = list()
31      for i in range(THREADS):
32          print(f'Spawning thread {i}')
33          t = threading.Thread(target=test_remote)
34          mythreads.append(t)
35          t.start()
36      for thread in mythreads:
37          thread.join()
```

Testing Remote Site (Locally Served WordPress Site)

1. So we are looping through the `web_paths` queue in line 17.
2. We are making the url that we suspect our target might have disclosed for us to initiate attack.
3. We sleep to give some time to our target so that we don't create a DOS attack situation.
4. In line 21 we are trying to get response from the suspected url.
5. Status code 200 means we got requested resource. Thus, the URL exists. We put the url inside answers queue since it is a valid url used by our target.

```
16 def test_remote():
17     while not web_paths.empty():
18         path = web_paths.get()
19         url = f'{TARGET}{path}'
20         time.sleep(2) # your target
21         r = requests.get(url)
22         if r.status_code == 200:
23             answers.put(url)
24             sys.stdout.write('+')
25         else:
26             sys.stdout.write('x')
27         sys.stdout.flush()
```

At last the MAIN

1. So at first we are changing the cwd using line 64 and executing `gather_paths()` so that our `web_paths` queue has all the paths, those might be disclosed by our target since our target is a wordpress site.
2. Next we execute `run()` to use threading while executing `test_remote()` definition.
3. While executing `test_remote()` we might find some paths those might be disclosed by our target. We will save those paths inside `answers` queue.
4. At last write the paths inside `answers` queue inside a text file.

```
63  if __name__ == '__main__':
64      with chdir(".\chapter5\wordpress\wordpress"):
65          gather_paths()
66          input('Press return to continue.')
67          run()
68      with open('.\chapter5\myanswers_nytco.txt', 'w') as f:
69          while not answers.empty():
70              f.write(f'{answers.get()}\n')
71      print('done')
```

MOST PROBABLY THIS IS THE LAST SLIDE

Some things to say,

1. Thank you for attending my course.
2. Sorry, to make you go through all the kali linux stuffs.
3. Sorry, to make you go through all the python codes.
4. In the future, whenever you face challenges with VMs or Python code, remind yourself: I conquered SWE 4503—this is just another hurdle I can easily overcome. Stay strong and keep pushing forward.